Homework before **Practical 01**

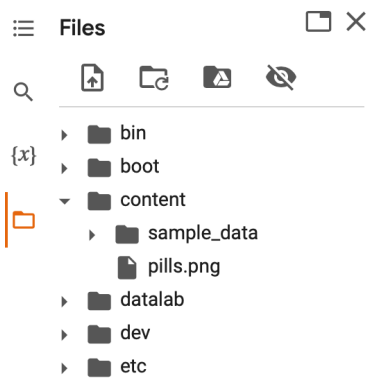
Credit: 50 points
Submit your Google Collab notebook link via email to Louisa **before the practical class meeting**.

1 Get familiar with Google Colab

This practical will again use Python, but we will switch 100% to Google Colab. If you are unfamiliar with Google Colab, take a look at their welcome page.

Start with creating an empty Colab notebook and sharing it with Walter (wscheire@nd.edu) and Louisa (lconwill@nd.edu) (or even simpler: with everyone at Notre Dame).

There are at least two ways to work with images in Google Colab. The first (simple) is to just upload image(s) you want to process (e.g., `pills.png` as in our homework) to the `/content/` folder you can see in the left panel in your Colab notebook:



The second (preferable) way is to mount your Google Drive folder and upload the images to Google Drive. Run the following code in your notebook to give Google Colab access to your Google Drive:

```
from google.colab import drive
drive.mount("/content/drive/")
```

All your Google Drive files will be visible in your notebook in `/content/drive/`. Note: neither Walter nor Louisa will have access to your Google Drive folder – we will only see your Python code. The upside of mounting your Google Drive is that the files won't disappear once you finish your current session with Colab.

2 What we will be doing in this practical?

Our goal in this practical session is to classify simple objects using their color, geometrical and texture features. This procedure will follow a general computer vision pipeline:

`Image -> Pre-processing -> Segmentation -> Feature extraction -> Classification`

Walter will bring a few images of simple objects of various colors and shapes to the class. We will use a Python package called `scikit-image` to calculate basic geometrical properties of objects. You can find

thetutorial and documents for this package at the scikit-image website. I would suggest to specifically look at what the `measure` module can calculate for you.

3 Task to do at home before Practical 01

In this homework I am asking you to write a Python program (from scratch – I am not providing any skeleton code this time) that will count automatically oval and round pills shown in this picture:

You should see 38 oval pills and 32 round pills in this image, and this is what your code should display. We will also need OpenCV (`cv2`), `numpy`, `matplotlib`, and a routine to display images in Colab (`cv2_imshow`). So the first lines of your notebook can look like this:

```
import cv2
from skimage import measure
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
```

Follow the steps in the next subsections. Once your code is ready, share your Colab notebook with everyone at the University of Notre Dame (don't forget about this step, otherwise we can't see your solution), and then paste the Colab notebook link in an email to Louisa as your "Practical 01 – at home" submission.

3.1 Pre-processing

The first step involves thresholding and converting the image to a binary image, as we did in Practical 00. The result will be a binary image with each object of interest expressed as a connected component of white pixels. Intensity of images that you will process in class may have some linear trend, thus its removal should help in getting a better binary image (do you remember one of the example dyadic point operators?). You may also see the need for histogram equalization before binarization. As an example, the binary image can be obtained in the following way, using the Fisher ratio-based thresholding approach (we use Otsu's implementation):

```
ret1, binary = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY+cv2.THRESH_OTSU)
```

3.2 Segmentation

Assuming that we already have a good binary image, your next task is to find all white blobs representing objects of interest. `scikit-image` package provides a very nice command to find all the connected components:

```
labels = measure.label(bw_img, n)
```

in which:

- `bw_img`: binary image
- `n`: connectivity

Note: `n` can take values of 1 or 2 (**not** 4 or 8), indicating either 4-connected or 8-connected components.

3.3 Feature extraction

In this homework I am asking you to use simple geometrical features to classify several classes of objects. For instance, looking for elliptic shapes we can model each localized object as an ellipse. The ratio of major axis to minor axis of the localized objects should be greater than 1 for ellipses, while for round objects the ratio should be roughly equal to 1. This example illustrates that even trivial features such as major-to-minor axes can help to classify simple objects.

Use a very useful `scikit-image` function `regionprops` to extract basic features from the connected components:

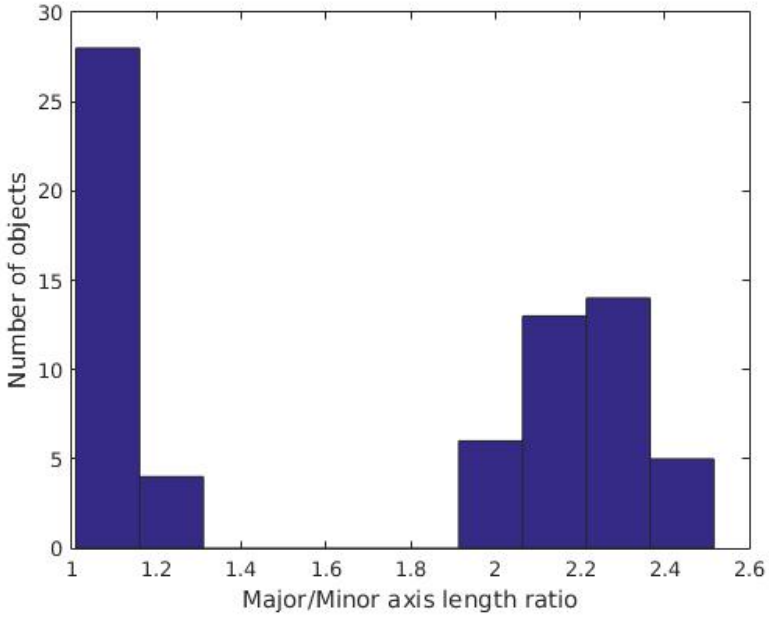
```
features = measure.regionprops(labels)
```

where:

- `labels` = connected components,
- `features` = comma-separated list of properties we want to extract, for example: 'area', 'bbox', 'centroid', ... For a complete list of properties check this webpage.

3.4 Classification

If we use `regionprops` to get major and minor axis length for each connected component, and plot the histogram of their ratio, we see that this single feature is sufficient to classify these shapes perfectly:



Certainly, one feature might not be enough to separate the classes, and you will see this in class. Using more features might lift up the data points to a higher-dimensional feature space in which they are well separated.

Now, when we have the features extracted, we can build a simple classifier to classify the objects: just check if the ratio is above, say, 1.5. If it is, the object is *elliptical*. If it is not, the object is *round*.